SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I S.Mahalakshmi(192524372) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
S.MAHALAKSHMI

QUESTION 1: Parsing Table Conflict in LL(1)

1.1 Scenario and Problem Statement

While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input-symbol combination. This creates uncertainty during parsing. The task is to explain how FIRST and FOLLOW sets are used, identify the source of the conflict, explain its effect on deterministic parsing, and recommend a grammar modification.

1.2 Relevant Concepts

- LL(1) parsing: Reads input from Left to right, constructs a Leftmost derivation, and uses one lookahead symbol.
- FIRST(X): The set of terminals that can appear first in strings derived from X. If X can derive ε, ε is also included.
- FOLLOW(A): The set of terminals that can occur immediately to the right of non-terminal A in a sentential form. The end marker $ is included in FOLLOW of the start symbol.
- Predictive parsing table: M[A,a] identifies the production to apply when A is on top of the stack and a is the next input token.

1.3 Detailed Analysis

Consider this example grammar:

$$S \rightarrow aA \mid aB$$
$$A \rightarrow b$$
$$B \rightarrow c$$

The FIRST sets of the two alternatives are identical:

$$FIRST(aA) = \{a\}$$
$$FIRST(aB) = \{a\}$$

When the table is constructed, both productions are placed in M[S,a], so the cell contains more than one production.

Non-terminal	Lookahead	Conflicting entries
S	a	S → aA and S → aB

FOLLOW sets are especially important when a production can derive ε. For a production A → α, FIRST(α) is used to enter the production in the relevant cells; if ε belongs to FIRST(α), FOLLOW(A) is also used. A conflict can occur when the resulting sets overlap.

1.4 Effect on Deterministic Parsing

An LL(1) parser must make its decision using only one lookahead symbol. If two productions occupy the same cell, the parser cannot uniquely select a production from the table. This is called a FIRST/FIRST conflict in the example. Other LL(1) conflicts can involve ε-productions and FOLLOW sets. The grammar is not LL(1) in its current form.

1.5 Decision and Proposed Solution

Apply left factoring to remove the common prefix a:

Original:
S → aA | aB
A → b
B → c

Left-factored:
S → aS′
S′ → A | B
A → b
B → c

After matching a, the parser can inspect the next token: b selects A, while c selects B. This removes the common-prefix conflict in the example. The entire grammar must still be checked for other LL(1) conflicts.

1.6 Conclusion

FIRST and FOLLOW sets guide predictive parsing-table construction. Multiple productions in one cell mean the parser cannot make a deterministic choice using one lookahead token. Left factoring is an appropriate decision when alternatives share a common prefix.

QUESTION 2: Incorrect Parse Tree Generation

2.1 Scenario and Problem Statement

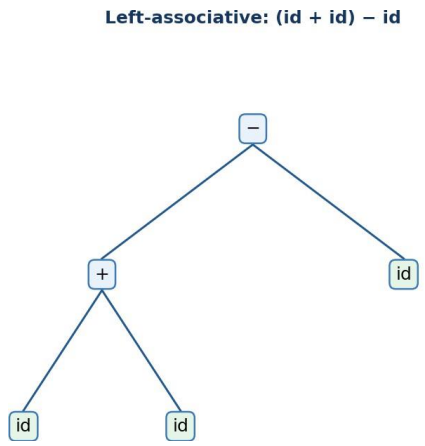
A syntax analyzer builds a parse tree for id + id - id, but evaluation is incorrect because the operators are grouped improperly. The scenario requires analysis of parse-tree construction, associativity, the effect of grammar rules, and a solution for correct grouping.

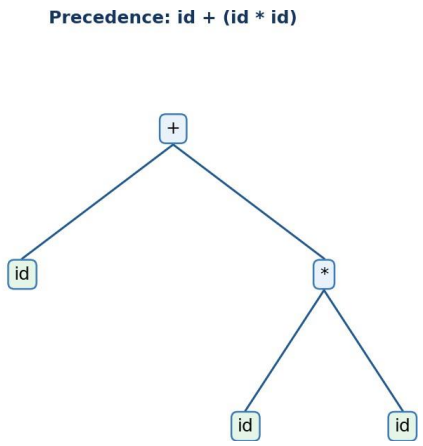
2.2 Relevant Concepts

- Parse tree: A hierarchical representation of how grammar productions derive the input.
- Associativity: A rule defining how operators of equal precedence are grouped.
- Left associativity: Groups operators from left to right, such as (a − b) − c.
- Ambiguous grammar: A grammar that permits more than one parse tree or derivation for a string.

2.3 Detailed Analysis

Assume + and − have equal precedence and are left-associative. The required grouping is (id + id) − id. A grammar such as E → E + E | E − E | id is ambiguous because it does not specify which operation should be the root of the parse tree.





The left tree shows the intended left-associative grouping. The second tree illustrates precedence grouping for a different expression, id + id * id, and is included to contrast the role of grammar structure.

2.4 Grammar and Decision

For a predictive parser, left recursion must be removed. A suitable simplified grammar for + and − is:

$$E \rightarrow T\ E'$$
$$E' \rightarrow +\ T\ E' \mid -\ T\ E' \mid \varepsilon$$
$$T \rightarrow id$$

This grammar recognizes a sequence of terms separated by + or −. Because the transformed grammar uses a tail E′, an evaluator should process the operations in the order they appear from left to right. For example, (10 + 4) − 2 = 12.

2.5 Conclusion

The parse tree reflects the grammar rules used by the parser. The grammar and semantic evaluation must preserve left associativity to interpret id + id − id as (id + id) − id.

QUESTION 3: Invalid Expression Handling

3.1 Scenario and Problem Statement

A lexical analyzer provides the token sequence for a + * b. Each token may be valid individually, but the syntax analyzer cannot generate a valid parse tree. The issue is the arrangement of tokens rather than whether each character is a known token.

3.2 Relevant Concepts

- Lexical analyzer: Converts characters into tokens such as identifiers, numbers, and operators.
- Syntax analyzer: Checks whether the token sequence conforms to grammar rules.
- Syntax error: A token is missing, unexpected, or placed in an invalid grammatical position.
- Error recovery: A parser strategy that attempts to continue after detecting an error.

3.3 Detailed Analysis

Consider a standard arithmetic grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow id$$

After reading a +, the parser expects an operand or a valid expression-starting token. Instead, it reads *. Since * is a binary operator and no operand appears between + and *, the token sequence violates the grammar.

3.4 Predictive Parsing and Shift-Reduce Parsing

Predictive parsing checks the table entry for the current non-terminal and lookahead token. If no production can be selected for the unexpected *, it reports a syntax error.

A shift-reduce parser uses a stack and performs shift and reduce operations. A shift places the next token on the stack; a reduction replaces a recognized right-hand side with its non-terminal. If no valid action is available for the current stack and input, the parser reports an error.

3.5 Decision: Detection and Recovery

A suitable decision is to provide a precise error message and use a recovery strategy such as panic-mode recovery. The parser skips input until a synchronization token, for example a semicolon or closing parenthesis, is found. It can then attempt to resume parsing.

```
Input: a + * b
Syntax Status: Invalid
Error: Unexpected '*' after '+'. An operand was expected.
```

3.6 Conclusion

The syntax analyzer detects that the multiplication operator appears where an operand is required. Predictive and shift-reduce parsers can report the error, and suitable recovery methods help prevent one invalid expression from stopping all subsequent analysis.

QUESTION 4: Ambiguity in Conditional Statements

4.1 Scenario and Problem Statement

The grammar S → if E then S | if E then S else S | a creates a conflict when nested conditional statements are parsed. The parser may be unable to determine which if statement should be associated with an else clause.

4.2 Relevant Concepts

- Ambiguous grammar: A grammar that permits multiple parse trees for the same input.
- Dangling-else problem: An else clause can appear to match more than one unmatched if.
- Matched statement: A conditional in which each if has a corresponding else.
- Unmatched statement: A conditional containing an if without a corresponding else.

4.3 Detailed Analysis

```
if E1 then if E2 then a else a
```

Interpretation 1: else belongs to the inner if. Interpretation 2: else belongs to the outer if. The original grammar does not explicitly specify which interpretation is intended, so the input can have more than one parse tree.

4.4 Decision and Proposed Solution

A common language convention is to associate each else with the nearest unmatched if. One standard approach is to distinguish matched and unmatched statements:

```
S → M | U
M → if E then M else M | a
U → if E then S
  | if E then M else U
```

M denotes matched statements and U denotes unmatched statements. Another practical solution is to use explicit delimiters, such as endif, or braces to mark the boundaries of conditional statements. The chosen grammar should be checked against the parser type and full language syntax.

4.5 Conclusion

The dangling-else problem is caused by ambiguity in conditional grammar. Separating matched and unmatched statements or using explicit delimiters clarifies the intended association and helps the parser generate the correct structure.

QUESTION 5: Operator Precedence Conflict

5.1 Scenario and Problem Statement

The grammar E → E + E | E * E | id is used to parse id + id * id. Since the grammar does not encode precedence between + and *, the syntax analyzer may build an incorrect parse tree.

5.2 Relevant Concepts

- Operator precedence: Determines which operator is grouped/evaluated first when different operators appear.

- Associativity: Determines grouping among operators with equal precedence.
- Ambiguous grammar: Allows more than one parse tree for a single expression.

5.3 Detailed Analysis

In ordinary arithmetic, multiplication has higher precedence than addition. Therefore, the intended expression is id + (id * id), not (id + id) * id.

For example, for 2 + 3 * 4, multiplication gives 3 × 4 = 12 and then addition gives 2 + 12 = 14. If addition were performed first, the result would be (2 + 3) × 4 = 20, which is not the standard interpretation.

5.4 Decision and Proposed Solution

Use separate grammar levels to encode precedence:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid id$$

E handles addition, T handles multiplication, and F handles identifiers or parenthesized expressions. For an LL(1) predictive parser, remove left recursion:

$$E \rightarrow T\ E'$$
$$E' \rightarrow + T\ E' \mid \varepsilon$$
$$T \rightarrow F\ T'$$
$$T' \rightarrow * F\ T' \mid \varepsilon$$
$$F \rightarrow (E) \mid id$$

The transformed grammar makes multiplication bind more tightly than addition. The grammar should still be checked for FIRST/FOLLOW conflicts before it is used as an LL(1) parser.

5.5 Conclusion

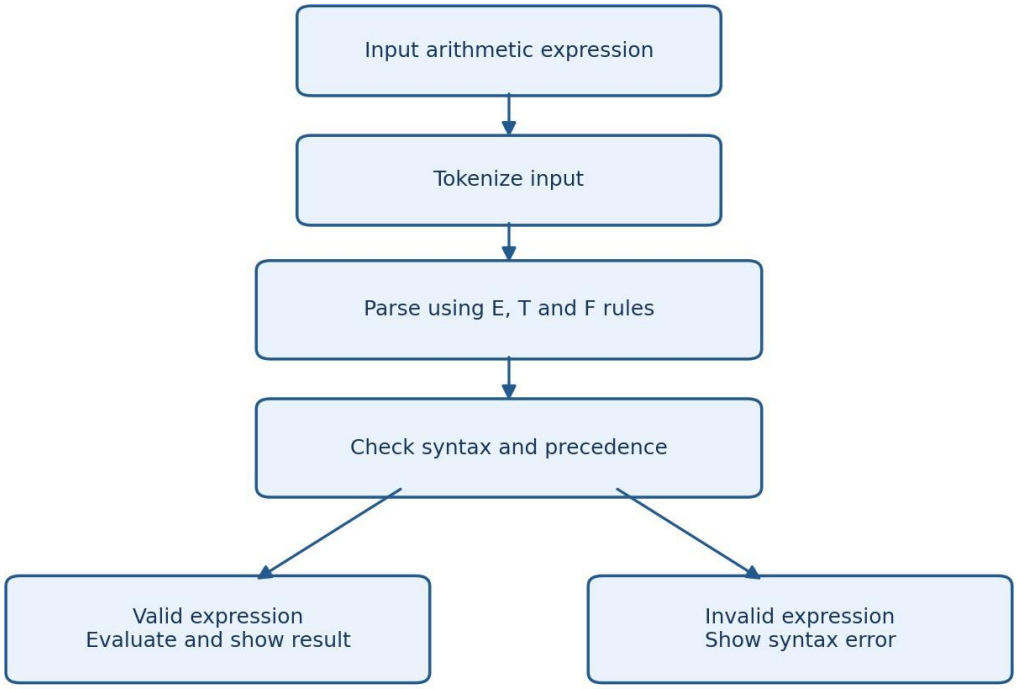
The original grammar does not distinguish operator precedence and is ambiguous. Separating expressions, terms, and factors ensures that multiplication is grouped before addition and provides a clear basis for correct evaluation.

PRACTICAL IMPLEMENTATION: MINI COMPILER SYNTAX ANALYZER

Purpose: This small educational implementation demonstrates arithmetic expression parsing, operator precedence, parentheses, and syntax-error reporting. It complements the theory in Questions 2, 3, and 5. It does not implement a complete compiler, a full LL(1) FIRST/FOLLOW generator, or a full dangling-else parser.

A. Implementation Overview

The program follows a recursive-descent approach. The grammar is divided into expression, term, and factor levels so that multiplication/division have higher precedence than addition/subtraction.



Educational implementation: arithmetic parsing, precedence and error reporting

B. Grammar Used in the Program

E → T ((+ | -) T)*
T → F ((* | /) F)*
F → number | (E)

E handles addition and subtraction. T handles multiplication and division. F handles a number or a parenthesized expression. This structure encodes precedence without relying on an ambiguous rule such as E → E + E | E * E.

C. Complete Python Implementation

```
import re

class Parser:
    def __init__(self, text):
        # Recognize numbers, parentheses, and arithmetic operators.
        self.tokens = re.findall(r"\d+(?:\.\d+)?|[()+\-*/]", text)
        self.position = 0

        # Reject unsupported characters instead of ignoring them.
        joined = "".join(self.tokens)
        compact = re.sub(r"\s+", "", text)
        if joined != compact:
            raise ValueError("Invalid character in expression.")

    def current(self):
        if self.position < len(self.tokens):
            return self.tokens[self.position]
        return None

    def eat(self, expected=None):
        token = self.current()
        if token is None:
            raise ValueError("Unexpected end of expression.")
        if expected is not None and token != expected:
            raise ValueError(
                "Expected '" + expected + "' but found '" + token + "'."
            )
        self.position += 1
        return token

    # E -> T ((+ | -) T)*
    def expression(self):
```

```
    value = self.term()
    while self.current() in ("+", "-"):
        operator = self.eat()
        right = self.term()
        if operator == "+":
            value = value + right
        else:
            value = value - right
    return value

# T -> F ((* | /) F)*
def term(self):
    value = self.factor()
    while self.current() in ("*", "/"):
        operator = self.eat()
        right = self.factor()
        if operator == "*":
            value = value * right
        else:
            if right == 0:
                raise ValueError("Division by zero.")
            value = value / right
    return value

# F -> number | ( E )
def factor(self):
    token = self.current()

    if token is None:
        raise ValueError("Expected an operand.")

    if token == "(":
        self.eat("(")
        value = self.expression()
        self.eat(")")
        return value
```

```
        if token in ("+", "-", "*", "/", ")"):
            raise ValueError(
                "Expected an operand, found '" + token + "'."
            )

        self.eat()
        return float(token) if "." in token else int(token)

    def parse(self):
        result = self.expression()
        if self.current() is not None:
            raise ValueError("Unexpected token '" + self.current() + "'.")
        return result


def main():
    text = input("Enter an expression: ")

    try:
        parser = Parser(text)
        result = parser.parse()
        print("Syntax Status: Valid")
        print("Precedence: * and / before + and -")
        print("Result:", result)
    except ValueError as error:
        print("Syntax Status: Invalid")
        print("Syntax Error:", error)


if __name__== "__main__":
    main()
```

D. Sample Inputs and Outputs

EXAMPLE 1:

Output

```
Enter an expression: 2+3*4
Syntax Status: Valid
Precedence: * and / before + and -
Result: 14
```

EXAMPLE 2:

Output

```
Enter an expression: (2+3)*4
Syntax Status: Valid
Precedence: * and / before + and -
Result: 20
```

EXAMPLE 3:

Output

```
Enter an expression: a+*b
ERROR!
Syntax Status: Invalid
Syntax Error: Invalid character in expression.
```

E. How to Run

1. Install Python 3 if it is not already installed.
2. Open Python IDLE or VS Code.
3. Create a file named mini_compiler.py.
4. Copy the complete code into the file and save it.
5. Run the program and enter an arithmetic expression when prompted.

F. Mapping to the Five Scenarios

Assignment scenario	How the implementation relates	Status
Q1: LL(1) parsing conflict	Explains the importance of unambiguous grammar; program does not build the full LL(1) table.	Theory only

Q2: Associativity	Processes + and − from left to right in the expression function.	Partially demonstrated
Q3: Invalid expression	Detects unexpected operators, missing operands, unmatched parentheses, and unsupported characters.	Implemented
Q4: Dangling else	Explained with matched/unmatched grammar; program does not parse if-then-else statements.	Theory only
Q5: Operator precedence	Uses separate expression, term, and factor functions to give * and / higher precedence.	Implemented

G. Limitations

- The program handles numeric arithmetic expressions, not variable names such as a and b.
- It does not generate a graphical parse tree at runtime.
- It does not automatically compute FIRST and FOLLOW sets or build a predictive parsing table.
- It does not implement conditional statements or dangling-else resolution.
- It is a teaching demonstration and is not a full compiler.

OVERALL CONCLUSION

The five scenarios show that grammar design influences parsing-table construction, parse-tree structure, syntax-error detection, conditional-statement interpretation, and arithmetic evaluation. FIRST and FOLLOW sets, left factoring, associativity, operator precedence, and error-recovery techniques are important for reliable syntax analysis. The included Python program provides a practical demonstration of expression precedence and syntax-error detection, complementing the theoretic.

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weigh tage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand